S

MIDTERM PROJECT DOCUMENTATION

Sawatâ: A Smart Gambling Blocker for Recovery

Advanced Mobile Programming

Student Name: _____

Section / Group: _____

Main Database: Firebase Cloud Firestore

Platform: Flutter (Android)

Prepared: August 19, 2026

Modules documented: lib/, android/app/src/main/kotlin/com/example/sawata/, firestore.rules

Description & Objectives

Project Description

Sawatâ is a Flutter/Android mobile application that helps people in gambling recovery avoid relapse by blocking gambling websites and apps at the device level, tracking recovery progress, and connecting each user with a trusted **Guardian** contact who can monitor that progress remotely. Website blocking runs through a local on-device VPN that filters DNS lookups against a blocklist; app blocking runs through an Accessibility Service that detects and closes flagged apps as they are opened. All user data (blocked items, journal entries, guardian relationships, recovery stats) is stored per-account in Firebase Cloud Firestore, protected by ownership-scoped security rules.

Project Objectives

- Block access to known gambling websites and apps in real time, without requiring the user to configure anything manually.
- Automatically flag newly opened apps/domains that look gambling-related (keyword matching + AI classification) and let the user confirm whether to block them.
- Give users a private space (journal) to record their mood and reflections during recovery.
- Let a user invite a trusted Guardian who can view recovery progress (streaks, blocked attempts, alerts) without being able to disable protection themselves.
- Keep all of the above backed by real, per-user cloud storage (Firebase Authentication + Firestore) instead of local/in-memory state, so progress survives reinstalls and is visible across devices.

PROJECT DOCUMENTATION — 2 OF 4

Scope, Limitations & System Architecture

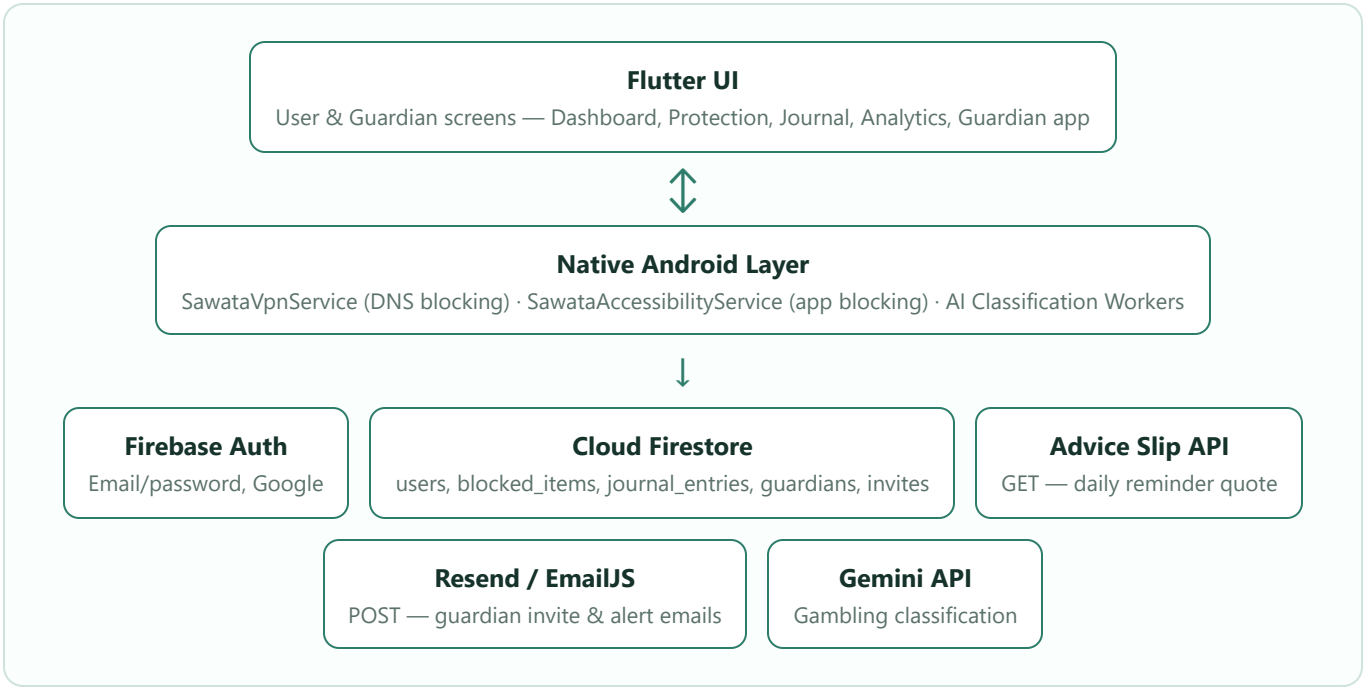
In Scope (Implemented)

- Android only — website blocking (local VPN) and app blocking (Accessibility Service) are native Android features with no iOS equivalent implemented.
- Email/password and Google sign-in via Firebase Authentication.
- Full CRUD on blocked sites/apps and journal entries, backed by Firestore.
- Guardian invite → accept flow, with the relationship persisted in Firestore and enforced by Firestore security rules.
- On-device keyword matching plus an AI classifier (Gemini, via a background Worker) that flags likely-gambling apps/domains for the user to confirm.
- Transactional email (guardian invites, security alerts) via Resend/EmailJS, with client-side rate limiting and retry.

Current Limitations

- The Guardian-side screens (Dashboard, Alerts, Reports, Settings) still render from in-memory sample data — not yet wired to a real Guardian's connected user(s) in Firestore.
- On the User Dashboard, headline stats (streak, blocked attempts) are real, but the weekly history chart still comes from seeded data.
- No iOS build target.
- No automated widget/integration tests yet — only unit tests for the email rate-limiter/retry/template logic exist.
- The "Advanced filters" button on Guardian Alerts is a stub.
- Journal entry deletion has no confirmation dialog (swipe-to-delete only).

System Architecture



Database Design (Firestore Schema)

Firestore is a document database, not relational — the tables below list each collection/subcollection, its parent, and the fields each document holds, which serves as this project's schema/ERD equivalent.

Collection	Path	Key Fields
users	users/{uid}	name, email, role, streakDays, blockedAttempts, protectionActive, protectedSince
blocked_items	users/{uid}/blocked_items/{itemId}	name, category, isBlocked, isCustom, packageName
journal_entries	users/{uid}/journal_entries/{entryId}	date, mood, title, body
suggestions	users/{uid}/suggestions/{packageName}	name, status
guardians	users/{uid}/guardians/{guardianUid}	name, relationship, email, status, connectedAt, permissions
invites	invites/{inviteId}	fromUid, toEmail, status
blocked_attempts	blocked_attempts/{attemptId}	uid, appName, timestamp
blocked_packages / blocked_domains	(top-level, admin-curated)	Global default blocklist
app_config	app_config/gambling_keywords	keywords[]
ai_candidates	(top-level)	AI-flagged items pending admin review
email_logs	(top-level)	Send log, used for rate limiting

Relationships

- **users → blocked_items, journal_entries, suggestions, guardians** — one-to-many, owned subcollections; access restricted to the owning `uid` by Firestore security rules.
- **invites** — references a sender (`fromUid`) and a recipient email; consumed by **guardians** writes to prove a valid invite chain.
- **blocked_attempts** — references a user by `uid`; queried by both that user and their connected guardian.

PROJECT DOCUMENTATION — 4 OF 4

Project Progress Report

Current Accomplishments

- Firebase Authentication (email/password + Google Sign-In) with role selection (User / Guardian).
- Full CRUD for blocked sites/apps and journal entries against Firestore, with live snapshot streams so the UI updates in real time.
- Firestore security rules enforcing per-user data ownership.
- Native Android DNS-level website blocking (local VPN) and Accessibility-Service-based app blocking.
- AI-assisted gambling detection for newly opened apps/domains, surfaced to the user as a confirm/dismiss suggestion.
- Guardian invite-and-accept flow, with transactional email delivery (Resend/EmailJS), client-side rate limiting, and retry logic.
- Search/filter on the Blocked Sites & Apps and Journal screens.
- Analytics screen with real blocked-attempt data and charts.

List of Completed Features

Feature	Status
User & Guardian authentication (email + Google)	**Complete**
Blocked sites/apps — Create, Read, Update, Delete	**Complete**
Journal — Create, Read, Update, Delete	**Complete**
Search/filter on Blocked Sites & Journal	**Complete**
Native VPN website blocking	**Complete**
Native Accessibility-Service app blocking	**Complete**
AI gambling-app/domain detection & suggestions	**Complete**
Guardian invite/accept flow + email delivery	**Complete**
Firestore security rules	**Complete**
User Dashboard real stats (streak, blocked attempts)	**Complete**

List of Remaining Features for Final Project

Feature	Status
Guardian Dashboard/Alerts/Reports/Settings wired to real Firestore data	In Progress
Weekly stat-history aggregation for Dashboard chart	Not Started
Advanced filters on Guardian Alerts	Not Started
Confirmation dialog for journal deletion	Not Started
Widget/integration test coverage	Not Started
Formal API documentation	Not Started

Honesty note: This document reflects what is actually implemented in the codebase as of August 19, 2026, verified against the source files directly (not just planned/aspirational features).